Bài tập lớn Xử lý ngôn ngữ tự nhiên (CO3085)

Trợ lý hỗ trợ học tập thông minh – Course Assistant

1 Giới thiệu

Xử lý ngôn ngữ tự nhiên (Natural Language Processing, NLP) đóng vai trò quan trọng trong việc xây dựng các hệ thống cho phép máy tính xử lý, phân tích và tương tác với ngôn ngữ của con người. Trong các hệ thống tương tác người - máy (Human-Computer Interaction, HCI), NLP cho phép người dùng giao tiếp với hệ thống bằng ngôn ngữ tự nhiên thay vì phải sử dụng các giao diện hoặc câu lệnh có cấu trúc cố định.

Các phương pháp NLP từ cổ điển đến hiện đại cung cấp nhiều kỹ thuật khác nhau để xử lý ngôn ngữ, bao gồm xây dựng văn phạm, phân tích cú pháp, biểu diễn ngữ nghĩa, nhận diện thực thể, phân loại văn bản, truy vấn thông tin và hỏi đáp. Bên cạnh đó, các phương pháp dựa trên học máy và các mô hình ngôn ngữ lớn (Large Language Models, LLMs) có thể được sử dụng để mở rộng khả năng của các hệ thống NLP.

Trong bài tập lớn này, sinh viên sẽ xây dựng một hệ thống **Trợ lý hỗ trợ học tập (Course Assistant)** trong một miền ngữ nghĩa hẹp. Hệ thống cho phép người dùng tương tác bằng ngôn ngữ tự nhiên để tra cứu và hỏi đáp về thông tin của một môn học.

Bài tập lớn được thiết kế nhằm giúp sinh viên vận dụng các kiến thức NLP đã học, đặc biệt là:

- văn phạm và phân tích cú pháp;
- biểu diễn và diễn giải ngữ nghĩa;
- nhận diện ý định và thông tin quan trọng trong câu;
- truy xuất thông tin và hỏi đáp;
- xử lý ngữ cảnh hội thoại đơn giản;
- các phương pháp NLP dựa trên luật hoặc học máy thống kê;
- mở rộng hệ thống bằng LLM/RAG ở mức cơ bản.

Sinh viên làm việc theo nhóm từ **3–4 sinh viên**.

2 Ngữ cảnh bài toán

Xét một hệ thống **Course Assistant** hỗ trợ sinh viên tra cứu và tương tác với thông tin của một môn học.

Hệ thống được cung cấp một tập dữ liệu/knowledge base của môn học, có thể bao gồm:

- thông tin chung của môn học;

- đề cương môn học;
- lịch học và kế hoạch giảng dạy;
- nội dung các chủ đề/bài học;
- thông tin bài tập;
- yêu cầu và thời hạn của bài tập lớn;
- quy định môn học;
- danh sách tài liệu tham khảo.

Người dùng có thể tương tác với hệ thống bằng các câu hỏi hoặc yêu cầu tự nhiên, ví dụ:

- "Môn NLP có bao nhiêu tín chỉ?"
- "Tuần 5 học những nội dung gì?"
- "Phần CFG và Parsing nằm ở tuần nào?"
- "Bài tập lớn có những yêu cầu gì?"
- "Deadline của bài tập lớn là khi nào?"
- "Tài liệu nào nói về CFG?"
- "Bài tập lớn có được sử dụng công cụ AI không?"
- "Tôi cần hoàn thành những bài tập nào?"

Ngoài các câu hỏi độc lập, hệ thống có thể hỗ trợ một số tương tác liên tiếp có sử dụng **ngữ cảnh cục bộ**, ví dụ:

> Sinh viên: "Tuần 5 học những gì?"
>
> Hệ thống: "Tuần 5 học về CFG và Parsing."
>
> Sinh viên: "Tài liệu của phần này ở đâu?"

Trong trường hợp trên, hệ thống cần sử dụng thông tin từ lượt tương tác trước để hiểu "phần này" đang đề cập đến nội dung vừa được trao đổi.

Phạm vi của hệ thống được giới hạn trong **miền thông tin của môn học được cung cấp**. Hệ thống không yêu cầu phải hiểu hoặc trả lời các câu hỏi tổng quát ngoài miền dữ liệu.

3 Phần I: Viết văn phạm và Parser

Trong phần này, sinh viên xây dựng các thành phần NLP cơ bản cho miền Course Assistant.

3.1 Viết tập rule và văn phạm cho miền Course Assistant

Sinh viên xây dựng một văn phạm đơn giản để mô tả các câu hỏi/yêu cầu thuộc miền môn học.

Văn phạm cần hỗ trợ tối thiểu một số nhóm câu hỏi/yêu cầu như:

- hỏi thông tin chung của môn học;
- hỏi nội dung/chủ đề;
- hỏi lịch học;
- hỏi thông tin bài tập;
- hỏi thời hạn;
- hỏi tài liệu;
- hỏi quy định;
- một số câu hỏi/yêu cầu liên quan khác trong phạm vi miền dữ liệu.

Input: dữ liệu nội bộ, ví dụ file grammar trong thư mục `data/` hoặc `models/`.
Output:

`output/grammar.txt`

3.2 Viết giải thuật sinh câu

Sử dụng văn phạm đã xây dựng để sinh các câu hợp lệ thuộc miền bài toán.
Yêu cầu:

- mỗi dòng tương ứng với một câu;
- câu được sinh phải phù hợp với văn phạm;
- giới hạn tối đa 10.000 câu.

Output:

`output/samples.txt`

3.3 Xây dựng bộ phân tích cú pháp

Xây dựng bộ phân tích cú pháp sử dụng văn phạm đã xây dựng.
Input:

`input/sentences.txt`

Mỗi dòng là một câu cần phân tích.
Output:

`output/parse-results.txt`

Đối với mỗi câu hợp lệ, hệ thống cần xuất ra cấu trúc cây cú pháp tương ứng. Nếu câu không hợp lệ hoặc không thuộc văn phạm, xuất:

`()`

4 Phần II: Diễn giải ngữ nghĩa và xây dựng Course Assistant

Dựa trên văn phạm và parser được xây dựng ở Phần I, sinh viên tiếp tục xây dựng hệ thống xử lý ngữ nghĩa và hỏi đáp.

Sinh viên lựa chọn **một trong hai hướng** sau:

1. **Lựa chọn 1: Classical/Statistical NLP**
2. **Lựa chọn 2: Mở rộng bằng LLM/RAG**

Cả hai lựa chọn đều phải sử dụng và thể hiện được kết quả của Phần I.

4.1 Lựa chọn 1: Classical/Statistical NLP

Sinh viên xây dựng hệ thống Course Assistant dựa trên các phương pháp NLP cổ điển và/hoặc phương pháp học máy thống kê.

Hệ thống cần thực hiện được chuỗi xử lý cơ bản:

```
Input
  -> Parsing
  -> Semantic Representation
  -> Intent/Entity
  -> Query
  -> Knowledge Base
  -> Answer
```

Hệ thống cần xử lý được một tập các loại truy vấn được xác định trước, ví dụ:

- GET_COURSE_INFO
- GET_TOPIC
- GET_SCHEDULE
- GET_ASSIGNMENT
- GET_DEADLINE
- GET_RESOURCE
- GET_RULE

Sinh viên thực hiện các bước sau:

1. **Xây dựng biểu diễn ngữ nghĩa**

   Ánh xạ câu tự nhiên sau khi phân tích cú pháp sang một biểu diễn ngữ nghĩa phù hợp.

   Ví dụ:

"Môn NLP có bao nhiêu tín chỉ?"

GET_CREDIT(course=NLP)

hoặc:

"Deadline bài tập lớn là khi nào?"

GET_DEADLINE(assignment=PROJECT)

Output:

output/semantic.txt

2. **Nhận diện intent và thông tin liên quan**

   Hệ thống xác định loại yêu cầu của người dùng và các thông tin cần thiết để truy vấn knowledge base.

   Sinh viên có thể sử dụng:

   - phương pháp dựa trên luật;
   - keyword matching;
   - phương pháp thống kê;
   - hoặc một mô hình học máy phù hợp.

   Không yêu cầu sử dụng Deep Learning hoặc LLM trong lựa chọn này.

   Output:

   output/intent-entity.txt

3. **Xây dựng truy vấn tới Knowledge Base**

   Từ semantic representation, xây dựng truy vấn tương ứng để tìm kiếm thông tin trong dữ liệu môn học.

   Output:

   output/query.txt

4. **Truy xuất thông tin và sinh câu trả lời**

   Hệ thống truy xuất thông tin từ knowledge base và tạo câu trả lời phù hợp cho người dùng.

   Output:

   output/answer.txt

5. **Xử lý trường hợp không tìm thấy thông tin**

   Nếu thông tin được hỏi không tồn tại trong knowledge base hoặc câu hỏi nằm ngoài phạm vi hệ thống, hệ thống cần đưa ra thông báo phù hợp thay vì tự tạo thông tin.

   Ví dụ:

   "Xin lỗi, tôi không tìm thấy thông tin này
   trong dữ liệu của môn học."

4.2 Lựa chọn 2: Mở rộng bằng LLM/RAG

Sinh viên có thể mở rộng hệ thống Course Assistant bằng mô hình ngôn ngữ lớn (Large Language Model, LLM) và/hoặc kỹ thuật Retrieval-Augmented Generation (RAG).

Lựa chọn này được xem là **phần mở rộng của NLP**, không phải yêu cầu bắt buộc của môn học.

Mục tiêu của phần này là giúp sinh viên tìm hiểu cách các phương pháp NLP hiện đại có thể được kết hợp với các thành phần NLP đã xây dựng ở Phần I.

Hệ thống tối thiểu cần thực hiện được:

User Question > Retrieve relevant information > Retrieved Context > LLM > Answer

Knowledge base có thể sử dụng cùng tập dữ liệu được cung cấp cho bài toán, ví dụ:

```
course/
    course-info.txt
    syllabus.txt
    schedule.txt
    assignments.txt
    regulations.txt
    resources.txt
```

Sinh viên thực hiện các bước sau:

1. Xây dựng hoặc sử dụng cơ chế truy xuất thông tin phù hợp.
2. Xây dựng prompt/context để cung cấp thông tin liên quan cho LLM.
3. Sinh câu trả lời dựa trên thông tin được truy xuất.
4. Nếu có thể, cung cấp nguồn/tài liệu được sử dụng để trả lời.
5. Xử lý trường hợp không tìm thấy thông tin phù hợp trong knowledge base.

Lưu ý:

Sinh viên **không bắt buộc** phải xây dựng:

- Agent hoặc Multi-Agent;
- hệ thống tool calling phức tạp;
- fine-tuning LLM;
- huấn luyện một mô hình ngôn ngữ;
- hệ thống RAG quy mô lớn;
- các kiến trúc Deep Learning phức tạp.

Sinh viên có thể sử dụng các mô hình hoặc thư viện công khai phù hợp, nhưng phải trình bày rõ cách thức sử dụng và hiểu được pipeline của hệ thống.

Output tối thiểu:

```
output/retrieval.txt
output/answer.txt
```

Trong đó cần thể hiện được:

- câu hỏi đầu vào;
- thông tin/tài liệu được truy xuất;
- câu trả lời của hệ thống.

5 *Phần III: Xử lý ngữ cảnh hội thoại cục bộ

Phần này nhằm mở rộng hệ thống để xử lý một số trường hợp tương tác liên tiếp có sử dụng thông tin từ lượt trước.

Ví dụ:

> Sinh viên: "Tuần 5 học những gì?"
>
> Hệ thống: "Tuần 5 học về CFG và Parsing."
>
> Sinh viên: "Tài liệu của phần này ở đâu?"

Hệ thống cần sử dụng ngữ cảnh cục bộ để xác định "phần này" đang đề cập đến nội dung vừa được trao đổi.

Sinh viên không yêu cầu xây dựng một Dialogue Manager phức tạp. Chỉ cần hỗ trợ một số trường hợp tham chiếu/ngữ cảnh được xác định trước.

Output:

`output/dialogue.txt`

File cần thể hiện được:

- hội thoại đầu vào;
- thông tin ngữ cảnh được sử dụng;
- kết quả diễn giải;
- câu trả lời.

6 Phần IV: Đánh giá hệ thống

Sinh viên xây dựng một tập dữ liệu kiểm thử để đánh giá hệ thống.

Tập kiểm thử cần bao phủ các loại truy vấn mà nhóm đã xây dựng và nên bao gồm:

- câu hỏi hợp lệ;
- các cách diễn đạt khác nhau của cùng một yêu cầu;
- câu hỏi không thuộc phạm vi;
- câu hỏi mà knowledge base không có thông tin;
- một số câu hỏi có sử dụng ngữ cảnh nếu nhóm thực hiện Phần III.

Sinh viên cần báo cáo tối thiểu:

- số lượng câu kiểm thử;
- số lượng câu trả lời đúng;
- độ chính xác hoặc chỉ số đánh giá phù hợp;
- một số trường hợp hệ thống trả lời sai;
- phân tích nguyên nhân của lỗi;
- nhận xét về ưu điểm và hạn chế của phương pháp đã sử dụng.

Nếu lựa chọn **LLM/RAG**, sinh viên cần bổ sung một số ví dụ về:

- câu trả lời đúng và có căn cứ;
- câu hỏi không có thông tin trong knowledge base;
- trường hợp hệ thống có nguy cơ sinh thông tin không có trong dữ liệu (hallucination).

Output:

`output/evaluation.txt`

7 Các yêu cầu khác

7.1 Ngôn ngữ và môi trường

Sinh viên sử dụng **Java hoặc Python** để thực hiện bài tập lớn.

Đối với lựa chọn sử dụng LLM/RAG, sinh viên có thể sử dụng các thư viện/framework/model phù hợp.

Không bắt buộc sử dụng một framework LLM/RAG cụ thể.

Bài làm phải có khả năng chạy trong môi trường được quy định và cần cung cấp đầy đủ hướng dẫn cài đặt/chạy chương trình.

Docker được sử dụng để hỗ trợ kiểm tra/chấm bài. Sinh viên cần nộp Dockerfile đầy đủ nếu được yêu cầu theo môi trường chấm.

7.2 Dữ liệu

Ban tổ chức/giảng viên cung cấp một knowledge base dùng chung cho bài toán.

Sinh viên có thể bổ sung dữ liệu phục vụ việc kiểm thử, nhưng phải:

- ghi rõ nguồn dữ liệu;
- không làm thay đổi bản chất bài toán;
- không sử dụng dữ liệu để tạo ra các câu trả lời ngoài phạm vi đã xác định.

7.3 Giới hạn sử dụng thư viện và công cụ

Sinh viên được phép sử dụng các thư viện NLP và Machine Learning công khai.

Đối với lựa chọn Classical/Statistical NLP, sinh viên không được sử dụng LLM để thay thế toàn bộ quá trình xử lý NLP cốt lõi.

Đối với lựa chọn LLM/RAG, sinh viên được phép sử dụng mô hình/ngưỡng dịch vụ phù hợp theo quy định của môn học, nhưng phải mô tả rõ:

- mô hình sử dụng;
- cách truy xuất thông tin;
- cách xây dựng prompt/context;
- cách đánh giá hệ thống.

7.4 Nộp bài

Sinh viên nộp file nén theo dạng:

```
MSSV1-MSSV2-MSSV3-MSSV4.zip
```

File nén gồm tối thiểu:

```
project/
    src/
    data/
    models/
    input/
    output/
    README.md
    Dockerfile
```

Trong đó:

- `src/`: mã nguồn chương trình;
- `data/`: dữ liệu cần thiết;
- `models/`: các model/configuration cần thiết nếu có;
- `input/`: dữ liệu đầu vào mẫu;
- `output/`: kết quả chạy thử;
- `README.md`: hướng dẫn cài đặt và chạy;
- `Dockerfile`: file cấu hình môi trường chạy.

File nén nên có kích thước nhỏ hơn **10MB**. Đối với các file/model lớn, sinh viên upload tới một nguồn lưu trữ phù hợp và cung cấp link trong `README.md`.

7.5 README.md

README cần mô tả tối thiểu:

- thành viên nhóm;
- hướng dẫn cài đặt;
- hướng dẫn chạy chương trình;
- mô tả kiến trúc hệ thống;
- mô tả grammar;
- mô tả parser;
- mô tả semantic representation;
- mô tả phương pháp xử lý intent/entity;
- mô tả knowledge base và cách truy vấn;
- mô tả lựa chọn Classical/Statistical NLP hoặc LLM/RAG;
- mô tả cách đánh giá;
- các hạn chế của hệ thống.

7.6 Báo cáo và Demo

Mỗi nhóm nộp:

- source code;
- README;
- output;
- **báo cáo bài tập lớn**;
- video demo ngắn.

Video demo cần thể hiện tối thiểu:

1. hệ thống nhận câu hỏi/yêu cầu;
2. quá trình xử lý chính;
3. kết quả phân tích hoặc semantic representation;
4. câu trả lời;
5. một trường hợp hệ thống không thể trả lời hoặc xử lý.

Đối với nhóm lựa chọn LLM/RAG, cần trình diễn thêm quá trình truy xuất thông tin và context được sử dụng để tạo câu trả lời.

Một số / hoặc toàn bộ các nhóm sẽ được lựa chọn để thuyết trình tại lớp. Thời gian dự kiến: tuần 13, 14 của học kỳ. Thời gian trình bày dự kiến của mỗi nhóm: 5-10 phút (sẽ có thông báo riêng sau).

8 Yêu cầu về tính trung thực học thuật

Sinh viên phải tự mình thực hiện bài tập lớn và hiểu rõ mã nguồn, thuật toán cũng như phương pháp được sử dụng.

Sinh viên được phép tham khảo:

- giáo trình và tài liệu môn học;
- tài liệu khoa học;
- thư viện và framework công khai;
- tài liệu hướng dẫn sử dụng;
- các công cụ AI theo quy định của môn học.

Mọi nguồn tham khảo, bao gồm sự hỗ trợ của AI, cần được kiểm tra và đảm bảo sinh viên hiểu và có khả năng giải thích lại.

Sinh viên **không nên sử dụng một hệ thống hoặc công cụ mà bản thân không hiểu cách thức hoạt động** để thay thế toàn bộ phần NLP được yêu cầu.

Đối với mã nguồn và tài liệu tham khảo từ bên ngoài, sinh viên phải ghi rõ nguồn.

Không được:

- sao chép code giữa các nhóm;
- chia sẻ toàn bộ hoặc một phần quan trọng của bài làm cho nhóm khác;
- sử dụng API hoặc hệ thống có sẵn để thay thế toàn bộ các thành phần NLP bắt buộc;
- trình bày kết quả hoặc mã nguồn mà thành viên nhóm không hiểu.

Mọi hành vi gian lận sẽ được xử lý theo quy định học vụ.

9 Yêu cầu về sản phẩm cuối cùng

Sản phẩm cuối cùng của mỗi nhóm phải thể hiện được một pipeline NLP hoàn chỉnh trong miền Course Assistant:

Natural Language Input > Grammar / Parser > Syntactic Representation > Semantic Interpretation > Intent / Entity > Knowledge Base / Retrieval > Answer

Nhóm có thể mở rộng:

Knowledge Base > Retrieval > LLM > Grounded Answer

nhưng phần mở rộng LLM/RAG không thay thế các yêu cầu NLP cốt lõi của bài tập lớn.

Mục tiêu cuối cùng của bài tập lớn là giúp sinh viên thể hiện khả năng **áp dụng kiến thức NLP để phân tích, xây dựng và đánh giá một hệ thống xử lý ngôn ngữ tự nhiên trong một miền ứng dụng cụ thể**.